

Justificación del Modelo de Persistencia (JPA) Sistema Polaflix

Sergio Lanza Teja

1 Introducción

El presente documento justifica las decisiones arquitectónicas tomadas en la implementación del modelo de dominio de Polaflix. La implementación se ha realizado utilizando el estándar JPA, respetando las fronteras de consistencia transaccional definidas por los principios de Domain-Driven Design (DDD).

2 Estrategia de Herencia

En nuestro modelo tenemos una herencia para los planes de pago: la clase padre `Suscripcion` y sus dos clases hijas, `SuscripcionFija` y `SuscripcionBajoDemanda`.

Para guardar esto en la base de datos, he elegido la estrategia de tabla única (`InheritanceType.SINGLE_TABLE`). Para saber de qué tipo es cada registro, usamos la columna (`@DiscriminatorColumn(name = "tipo_suscripcion")`).

- **Mejor rendimiento (Cero JOINS):** Al meter toda la jerarquía en una sola tabla, cuando queremos cargar a un usuario y su suscripción solo hace falta un `SELECT` básico. Así nos ahorramos hacer operaciones `JOIN` más pesadas que sí tendríamos que hacer con otras estrategias.
- **Pocos campos nuevos en los hijos:** Las clases hijas casi no añaden atributos nuevos. `SuscripcionFija` solo suma el campo `cuotaMensual` y la otra no añade nada. Aunque esto deje algunos huecos a `NULL` en la tabla, compensa de sobra por lo rápido y sencillo que queda el esquema.

3 Elección de Identificadores (Claves Primarias)

Para las claves primarias de todas las entidades, he decidido usar IDs artificiales (subrogados) con la anotación `@Id` y `@GeneratedValue(strategy = GenerationType.AUTO)`.

- **Independencia de la base de datos:** Al usar `GenerationType.AUTO`, le dejamos el trabajo a JPA (Hibernate). Él mismo decide la mejor forma de generar los números (usando secuencias, autoincrementos, etc.) dependiendo del motor de base de datos que estemos usando por debajo.
- **Separar el negocio de la base de datos:** Al usar IDs numéricos propios (como `idUsuario` o `idSerie`) en lugar de usar datos reales (como el nombre de usuario o el título), nos quitamos problemas futuros. Si un usuario decide cambiarse el nombre en la aplicación, la base de datos no sufre en absoluto, porque todas las relaciones y claves foráneas siguen apuntando al mismo ID, que nunca cambia.

4 Mapeo de Relaciones y Políticas de Cascada

El diseño de las relaciones y la propagación de operaciones (*cascading*) se ha modelado estrictamente en función de la pertenencia a los *Aggregate Roots* (Raíces de Agregado).

4.1 Asociación con cascada (Ciclo de Vida Acoplado)

Para las relaciones donde una entidad débil depende existencialmente de su raíz de agregado, se ha aplicado `cascade = CascadeType.ALL` y `orphanRemoval = true`.

- **Catálogo** (`Serie` → `Temporada` → `Capitulo`): Las temporadas y capítulos no tienen sentido fuera de su serie. Si se elimina una `Serie`, el borrado se propaga en cascada eliminando toda su estructura interna de la base de datos.
- **Historial** (`Usuario` → `ProgresoSerie` → `Visualizacion`): El `ProgresoSerie` es una entidad débil del usuario. Al activar `orphanRemoval = true`, garantizamos que si se elimina un progreso de la lista del usuario (o una visualización del progreso), el registro huérfano es purgado automáticamente de la base de datos, evitando inconsistencias.
- **Facturación** (`Factura` → `EntradaFactura`): La factura gobierna sus líneas de cobro. Se controla el ciclo de vida de las entradas a través de la raíz del agregado contable.

4.2 Asociaciones sin Cascada (Independencia)

Cuando dos entidades pertenecen a *Aggregate Roots* distintos, no se aplica propagación de cascada.

- `Usuario` y `Serie`: Las relaciones de listas como `seriesPendientes` o `seriesTerminadas` están mapeadas con `@ManyToMany` puro, sin *cascade* ni remoción de huérfanos. El catálogo de series y los usuarios tienen ciclos de vida independientes. Si un usuario se da de baja y se borra de la base de datos, las series que estaba viendo no deben verse afectadas.
- `Usuario` y `Factura`: Aunque la factura pertenece al usuario, la colección de facturas mapeada en `Usuario` no posee *cascade*. Por requisitos de auditoría y obligaciones fiscales, una `Factura` es un documento histórico persistente. Si se elimina el `Usuario`, sus facturas emitidas deben conservarse intactas en el sistema.

4.3 Optimización Bidireccional (`mappedBy`)

En las relaciones bidireccionales de tipo `@OneToMany`, se ha definido sistemáticamente la entidad del lado "Muchos" como la propietaria de la relación, utilizando el atributo `mappedBy` en el lado "Uno" (ej. `mappedBy = "serie"` en la clase `Serie`). Esto instruye a JPA para que aloje una clave foránea limpia en la tabla dependiente, evitando la creación de costosas e innecesarias tablas intermedias (*join tables*).

5 Otras Consideraciones Técnicas

- **Mapeo de Enumerados**: Para propiedades de dominio clasificatorias, como el `TipoSerie`, se ha empleado `@Enumerated(EnumType.STRING)`. Al almacenar el valor textual en lugar del índice ordinal numérico (`EnumType.ORDINAL`), se previene la corrupción de datos en la base de datos si en el futuro se refactoriza el orden de los valores del *enum* en el código Java.